

Review Questions and Exercises

Short Answer

1. What does the indirection operator do?

2. Look at the following code.

```
int x = 7;
int *iptr = &x;
```

What will be displayed if you send the expression `*iptr` to `cout`? What happens if you send the expression `ptr` to `cout`?

3. So far you have learned three different uses for the `*` operator. What are they?

4. What math operations are allowed on pointers?

5. Assuming `ptr` is a pointer to an `int`, what happens when you add 4 to `ptr`?

6. Look at the following array definition.

```
int numbers[] = { 2, 4, 6, 8, 10 };
```

What will the following statement display?

```
cout << *(numbers + 3) << endl;
```

7. What is the purpose of the `new` operator?

8. What happens when a program uses the `new` operator to allocate a block of memory, but the amount of requested memory isn't available? How do programs written with older compilers handle this?

9. What is the purpose of the `delete` operator?

10. Under what circumstances can you successfully return a pointer from a function?

11. What is the difference between a pointer to a constant and a constant pointer?

12. What are two advantages of declaring a pointer parameter as a constant pointer?

Fill-in-the-Blank

13. Each byte in memory is assigned a unique _____.

14. The statement `ptr = &num` assigns the _____ of `num` to the pointer `ptr`.

15. The statement `cout << *(&num)` prints the _____ of the variable `num`.

16. A(n) _____ can hold the address of an integer variable.

17. Array names can be used as _____, and vice versa.

18. Creating variables while a program is running is called _____.

19. The _____ operator is used to dynamically allocate memory.

20. Under older compilers, if the `new` operator cannot allocate the amount of memory requested, it returns _____.

21. A pointer that contains the address 0 is called a(n) _____ pointer.

22. The _____ operator is used to release the amount of memory that was dynamically created using the _____ operator.

23. You should only use pointers with `delete` that were previously used with _____.

Algorithm Workbench

24. Look at the following code:

```
double value = 29.7;
double *ptr = &value;
```

Write a `cout` statement that uses the `ptr` variable to display the contents of the `value` variable.

25. Look at the following function call.

```
iptr = someFunc ( x, &y);
```

If `x` and `y` are integers and `iptr` is a pointer to an integer, write the function prototype.

26. Write code to dynamically allocate an integer array, allow the user to input the array elements, find their mean, and finally release the memory used by the array.
27. Write a single statement that displays the value pointed by a pointer `xptr`, and then increments the pointer `xptr`.
28. Look at the following function definition:

```
void assign ( int *numbers, int size )
{
    for ( int i = 0 ; i < size ; i++ )
        cin >> numbers[i];
}
```

Write code showing how you can call this function. Also, rewrite the `cin` statement to show an alternative way to input elements in the array.

29. Write the definition of `ptr`, a pointer to a constant `int`.
30. Write code to display a two-dimensional array `arr[5][10]`, using pointer notation.

True or False

31. T F Each byte of memory is assigned a unique address.
32. T F The `*` operator is used to get the address of a variable.
33. T F A pointer variable can be incremented and decremented.
34. T F The `&` symbol is called the indirection operator.
35. T F The statement `const double *dptr` indicates that `dptr` is a constant pointer.
36. T F When the indirection operator is used with a pointer variable, you are actually working with the value the pointer is pointing to.
37. T F Array names cannot be dereferenced with the indirection operator.
38. T F When you add a value to a pointer, you are actually adding that number times the size of the data type referenced by the pointer.
39. T F A constant pointer implies a pointer to a constant variable.
40. T F You can change the address that an array name points to.
41. T F Any mathematical operation, including multiplication and division, may be performed on a pointer.
42. T F Once a constant pointer is initialized with an address, it cannot point to anything else.
43. T F Though a function can accept pointer variables as arguments, a function can never return a pointer.

- 44. T F The new operator dynamically allocates memory.
- 45. T F A pointer variable that has not been initialized is called a null pointer.
- 46. T F The address 0 is generally considered unusable.
- 47. T F A smart pointer can automatically delete a dynamically allocated memory when it is not in use anymore, without the need of the delete operator to be explicitly called to do so.

Find the Error

Each of the following definitions and program segments has errors. Locate as many as you can.

- 48. `double *dptr = *nullptr;`
- 49. `int x, *ptr = nullptr;`
`&x = ptr;`
- 50. `int ptr, value=9;`
`ptr = &value;`
- 51. `int x, *ptr = nullptr;`
`ptr = &x;`
`ptr = 100; // Store 100 in x`
`cout << x << endl;`
- 52. `float numbers[] = { 2.3, 3.4, 4.5, 5.6, 6.7};`
`cout << "The second element in the list of numbers is :";`
`cout << (*numbers++) << endl;`
- 53. `int values[20], *iptr = nullptr;`
`iptr = values;`
`iptr *= 2;`
- 54. `float level;`
`int fptr = &level;`
- 55. `float values[6] = {1,2,3,4,5,6};`
`cout << *(values + 1.5) << endl;`
- 56. `void doubleVal(int val)`
`{`
`*val *= 2;`
`}`
- 57. `int *pint = nullptr;`
`new pint;`
- 58. `int *pint = nullptr;`
`pint = new int;`
`if (pint == nullptr)`
`*pint = 100;`
`else`
`cout << "Memory allocation error\n";`

```

59. int *pint = nullptr;
    pint = new int[100]; // Allocate memory
    :
    Code that processes the array.
    :
    delete pint; // Free memory
60. int *getNum()
    {
        int wholeNum;
        cout << "Enter a number: ";
        cin >> wholeNum;
        return &wholeNum;
    }
61. const int arr[] = { 1, 2, 3 };
    int *ptr = arr;
62. void doSomething(int * const ptr)
    {
        int localArray[] = { 1, 2, 3 };
        ptr = localArray;
    }

```

Programming Challenges

1. Address of Elements in an Array

Write a program to initialize an array of integers with 10 numbers. The program should then display the number of bytes occupied by the array. Also display each element as well as the address in which it is stored.

2. Test Scores #1

Write a program that dynamically allocates an array large enough to hold a user-defined number of test scores. Once all the scores are entered, the array should be passed to a function that sorts them in ascending order. Another function should be called that calculates the average score. The program should display the sorted list of scores and averages with appropriate headings. Use pointer notation rather than array notation whenever possible.

Input Validation: Do not accept negative numbers for test scores.

3. Drop Lowest Score

Modify Problem 2 above so the lowest test score is dropped. This score should not be included in the calculation of the average.